

2 차 추가 회의 정리 초안

회의자: 이정수, 김진비, 최준환

메모: 이 문서는 팀원 8명 전체가 공식적으로 확정된 회의록은 아니고, 전체 회의 이후 3명이 남아서 추가로 이야기한 내용을 정리한 초안이다. 그래서 최종 결정이라기보다는 다음 전체 회의에서 다시 확인하고 조정해야 할 내용으로 보면 될 것 같다.

1. 일단 방향성

일단 우리가 만드는 게 단순 게시판인지, 아니면 나중에 확장 가능한 에셋 관리 서비스인지 먼저 생각해봐야 할 것 같다.

현재 요구사항만 보면 게시물 작성, 파일 업로드, 카테고리 정도만 있어도 될 것 같지만, 통합 프로젝트까지 생각하면 처음부터 어느 정도 확장 가능성을 염두에 두는 게 맞는 것 같다.

그래서 모든 기능을 처음부터 다 만들자는 의미는 아니고, 최종적으로 만들 수 있는 서비스 모습을 어느 정도 상상한 다음 그중에서 최소 기능만 MVP로 가져가는 방향이 좋아 보인다.

2. 우리가 이해한 클라이언트 요구사항

1. 360도 돌아가는 에셋 확인 기능 - 프론트 비중이 클 것 같음
2. 디렉토리 / 카테고리 분류 문제 - 태그나 자동 추천으로 보완 필요
3. 에셋 요청 게시판 - 다른 팀에게서 요구서를 받고 구현하는 경험 필요

2.1 360도 에셋 확인 기능

에셋을 단순 이미지나 파일로만 보여주는 게 아니라, 사용자가 360도로 돌려보면서 확인할 수 있는 기능이 필요해 보인다. 이 부분은 백엔드보다는 프론트 쪽 구현 비중이 꽤 클 것 같다.

2.2 디렉토리 / 카테고리 분류 문제

기존 서비스에서 카테고리 분류가 엉망이라는 요구사항이 있었고, 이게 생각보다 어려운 문제라는 이야기를 했다.

실제 에셋: 도쿄타워
제목: 계란
카테고리: 자연
태그: 음식

이 경우 검색과 카테고리 탐색이 둘 다 망가질 수 있다. 계란을 검색했는데 도쿄타워가 나오고, 자연 카테고리에 들어갔는데 도쿄타워가 나오는 식이다.

2.3 에셋 요청 게시판

다른 팀에서 필요한 에셋을 요청할 수 있는 게시판도 필요해 보인다. 약간 GitHub Issue 처럼 요청이 올라오고, 담당자가 수락하고, 완료되면 아카이브되는 흐름으로 생각했다.

요청글 작성
↓
담당자 수락 / assignee 배정
↓
에셋 제작
↓
에셋 업로드
↓
요청 완료 처리
↓
아카이브

3. MVP로 가져갈 수 있는 것

일단 MVP는 너무 복잡하게 잡지 말고, 아래 정도를 최소 기능으로 생각해볼 수 있을 것 같다.

- 로그인

- 기본 홈 화면
- 카테고리 대분류
- 에셋 요청 게시판
- 게시글 목록
- 게시글 작성
- 개인 profile / 포트폴리오

게시글 작성에는 최소한 아래 정보가 필요할 것 같다.

- 제목
- 내용
- 파일 업로드
- 태그

여기에 나중에 팀 단위 전환을 생각하면 teamId 같은 값도 미리 고려해두는 게 좋을 것 같다.

4. 걱정되는 부분

4.1 도메인별 업무량 불균형

도메인별로 역할을 나누면 깔끔하긴 한데, 실제 업무량이 균등하지 않을 수 있다. 특히 인프라 쪽은 배포 이후에 오류나 트래픽 문제가 생기면 일이 갑자기 많아질 수 있을 것 같다. 그래서 처음 역할을 나누더라도 배포 후에는 다시 한번 리밸런싱이 필요할 것 같다.

4.2 프론트 비중이 생각보다 큼

처음에는 Thymeleaf로 생각할 수 있지만, 실제 React로 구현한다고 생각하면 프론트가 차지하는 시간과 중요도가 제일 클 수도 있을 것 같다.

- 360도 에셋 뷰어
- 카테고리 대분류 / 중분류 / 소분류 UI
- 썸네일 기반 카드 UI
- 검색 결과 화면
- 개인 포트폴리오 페이지
- 요청 게시판 상태 UI
- 인터랙티브 에셋 미리보기

4.3 카테고리 분류 문제

이 부분은 오늘 논의 결과, GPT API 같은 것을 이용해서 자동 추천을 붙이는 방향이 괜찮지 않을까 하는 쪽으로 이야기가 나왔다. 다만 MVP에서 어디까지 할지는 다시 정해야 한다.

방법 1. 수동 검수

사용자가 업로드한 에셋을 관리자가 직접 확인하고 카테고리나 태그를 수정하는 방식이다. 정확도는 높지만 업로드 수가 많아질수록 관리 비용이 너무 커져서 비효율적일 수 있다.

방법 2. AI 기반 자동 태그 생성

사용자가 파일이나 썸네일을 업로드하면 AI가 이미지나 설명을 기반으로 자동 태그를 생성하는 방식이다.

예시 태그:
도쿄타워
건축물
도시
일본
랜드마크
야경

사용자가 제목이나 카테고리를 잘못 입력하더라도 태그 검색으로 어느 정도 보완할 수 있다는 장점이 있다. 단점은 GPT API 또는 이미지 분석 API 사용 비용이 발생할 수 있다는 점이다.

방법 3. AI 기반 카테고리 추천

게시글 작성 시 AI가 썸네일이나 파일명을 보고 적절한 카테고리를 추천하는 방식이다.

선택한 카테고리: 자연

AI 추천 카테고리:

- 건축물
- 도시
- 랜드마크

추천 카테고리로 변경하시겠습니까?

사용자의 선택권은 유지하면서도 더 정확한 분류를 유도할 수 있다는 점이 괜찮아 보인다.

방법 4. 충돌 발생 시 관리자 검토 요청

사용자가 선택한 카테고리와 AI 추천 카테고리가 다를 경우 관리자 검토 목록에 올리는 방식이다.

사용자 선택 카테고리: 자연

AI 추천 카테고리: 건축물

- 카테고리 충돌 발생
- 관리자 검토 필요 상태로 변경

개인적으로는 자동화와 수동 검수의 중간 지점이라 가장 괜찮아 보인다. 다만 MVP 단계에서 바로 넣기에는 조금 무거울 수 있다.

4.4 개인 단위에서 팀 단위로 바뀌는 문제

현재는 개인 사용자가 로그인해서 에셋을 업로드하고, 개인 프로필에 포트폴리오처럼 쌓는 구조를 생각하고 있다. 그런데 통합 프로젝트가 시작되면 개인 단위가 아니라 팀 단위로 작업이 진행될 가능성이 높다. 문제는 이때 구조를 갑자기 바꾸면 큰 수정이 필요할 수 있다는 점이다.

프론트에서 팀 1, 팀 2, 팀 3으로 나누는 방식

프론트에서 단순히 카테고리를 팀 1, 팀 2, 팀 3으로 나누는 방식은 겉으로는 쉬워 보이지만, 데이터 구조 자체가 팀을 이해하는 게 아니라서 확장성이 떨어질 것 같다.

User에 teamId를 미리 포함하는 방식

처음부터 User에 teamId를 nullable 값으로 넣어두는 방식이 괜찮아 보인다.

초기 개인 프로젝트 단계:

```
User.teamId = null
```

팀 프로젝트 전환 이후:

```
User.teamId = 1
```

```
User.teamId = 2
```

```
User.teamId = 3
```

게시글을 작성할 때도 작성자의 teamId를 같이 저장하면 나중에 팀 단위로 전환할 때 구조 변경이 훨씬 쉬워질 것 같다. 이건 꽤 괜찮은 방향으로 보인다.

```
Post
- id
- title
- content
- userId
- teamId
- categoryId
- createdAt
```

팀 프로젝트 전환 후에는 현재 로그인한 사용자의 teamId를 기준으로 같은 팀 게시글만 조회할 수 있다.

현재 로그인한 사용자 teamId = 2

게시글 목록 조회

→ teamId = 2인 게시글만 조회

또 공개 범위를 추가하면 더 유연하게 갈 수 있다.

```
visibility = PUBLIC
visibility = TEAM_ONLY
visibility = PRIVATE
```

```
PUBLIC      → 모든 사용자 조회 가능
TEAM_ONLY  → 같은 teamId 사용자만 조회 가능
PRIVATE    → 작성자 본인만 조회 가능
```

5. 디렉터리 구조

선생님께서도 추후 확장 가능성을 생각하면 MSA 느낌의 디렉터리 구조로 가는 게 좋겠다고 말씀하셨다. 실제로 서비스를 여러 개로 쪼개는 MSA는 아니더라도, 패키지를 도메인별로 나누는 방식이다.

```
user
├── controller
├── service
├── repository
├── dto
└── entity

category
├── controller
├── service
├── repository
├── dto
└── entity

post
├── controller
├── service
├── repository
├── dto
└── entity

request / comment / file / tag / team / infra / frontend
```

장점은 각 섹션별 책임 구조가 명확하다는 점이다. 문제가 생겼을 때 전체 도메인을 다 보는 것이 아니라 문제 발생 도메인만 집중해서 볼 수 있다.

단점은 각자 맡은 도메인만 보게 되면서 커뮤니케이션이 줄어들 수 있다는 점이다. 도메인 간 연결 지점에서 충돌이 생길 수도 있고, 공통 규칙을 정하지 않으면 코드 스타일이 달라질 수 있다.

그래서 도메인 구조를 나누더라도 아래 내용은 먼저 맞추는 게 좋을 것 같다.

- DTO 이름 규칙
- API 응답 형식
- 예외 처리 방식
- soft delete 사용 여부
- createdAt / updatedAt 처리 방식
- 권한 처리 방식

6. 도메인 분류 초안

도메인	대략적인 역할
User	로그인, 회원가입, 권한, 프로필, teamId 관리
Team	팀 생성, 팀원 배정, 팀별 게시글/요청글/포트폴리오 조회
Category	대분류/중분류/소분류 조회, 생성, 수정, 삭제, 관리 페이지
Post	홈 화면, 개인 포트폴리오, 게시글 작성/상세/목록, 에셋 소개 화면
Request	에셋 요청 게시판, assignee 배정, 상태 관리, 완료/아카이브
Comment	Post 댓글과 Request 댓글을 하나로 처리하는 구조 검토
File	파일 업로드, 다운로드, 썸네일, 게시글/요청글과 파일 연결
Tag + Search	태그 입력/검색, AI 추천 태그, Elasticsearch 확장
Infra	로컬 세팅, Docker, MySQL, Redis, CI/CD, 배포, 모니터링
Frontend	360도 뷰어, 카테고리 UI, 카드 UI, 프로필, 요청 게시판 UI

6.1 User

- security 포함
- Admin / Super 권한 구분
- 비밀번호, 이메일, 프로필 정보
- teamId 관리

6.2 Category

- 홈 화면 왼쪽에 대분류 나열
- 대분류 선택 시 전체 글 화면 위쪽에 중분류 가로 형식으로 나열
- 중분류 선택 전에는 소분류 영역만 비어 있음
- 중분류 선택 시 소분류 화면에 표시
- 카테고리 조회, 수정, 삭제 페이지로 랜딩

6.3 Post

- 홈 화면
- 개인 portfolio 페이지 화면
- 게시글 작성 화면
- 요청 게시판 화면
- 요청 게시글 화면
- 각 에셋별 소개 화면

6.4 Comment

Post comment와 Request comment가 비슷한 기능이라 하나로 묶을 수 있을 것 같다. 처음부터 상속으로 복잡하게 가기보다는 targetType, targetId로 구분하는 방식도 괜찮아 보인다.

```
Comment
- id
- content
- writerId
- targetType
- targetId
- createdAt
- updatedAt
- deletedAt
```

6.5 Request 게시판

일단 블로그 게시판의 하위호환처럼 보이지만, 여기에 assignee 배정, 해결 처리, 아카이브 관리가 들어간다는 점에서 차이가 있다.

```
요청 게시판에 유니티 반이 에셋 요청 올림
↓
assignee가 수락 후 1대1로 연결됨
↓
에셋 업로드
↓
연동된 요청글 완료/soft delete 처리
↓
나중에 로그 분석을 위해 아카이브
```

6.6 File

- 게시글 작성 시 파일 업로드
- 썸네일 구현
- 파일과 게시글 연결

6.7 Tag + ElasticSearch

카테고리 분류가 틀릴 수 있기 때문에 태그 검색 기능이 보조 역할을 할 수 있을 것 같다. 확장하면 API 를 사용해서 게시글 파일 업로드 시 썸네일 기반으로 자동 태그나 자동 설명을 생성할 수도 있다.

6.8 Infra

- 로컬 세팅
- CI/CD
- Docker
- Redis
- MySQL
- Prometheus + Grafana
- ElasticSearch + Kibana
- 초기 설정 및 배포 후 관리

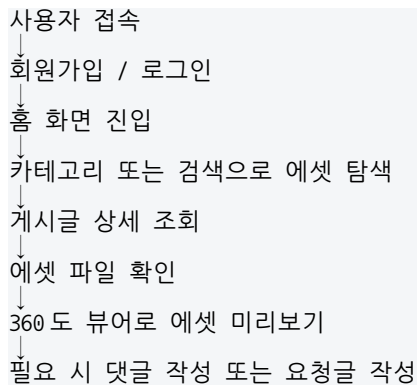
참고로 Keavana 라고 적은 부분은 아마 Kibana 를 의미하는 것 같다.

6.9 Frontend

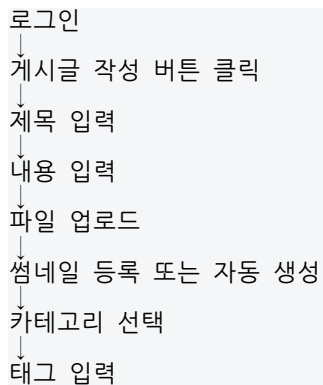
- 360 도 에셋 뷰어
- 카테고리 대분류 / 중분류 / 소분류 UI
- 썸네일 기반 카드 UI
- 검색 결과 화면
- 개인 포트폴리오 페이지
- 요청 게시판 상태 UI
- 인터랙티브 에셋 미리보기

7. 기본 사용자 플로우

7.1 기본 에셋 조회 흐름



7.2 에셋 업로드 플로우



↓
게시글 공개 범위 선택
↓
작성 완료
↓
게시글 목록 및 개인 포트폴리오에 반영

7.3 AI 태그 추천 확장 플로우

파일 업로드
↓
썸네일 생성
↓
AI가 썸네일 분석
↓
추천 태그 생성
↓
사용자에게 추천 태그 표시
↓
사용자가 태그 선택 또는 수정
↓
게시글 저장

7.4 요청 게시판 플로우

요청자 로그인
↓
요청 게시판 이동
↓
에셋 요청글 작성
↓
상태: OPEN
↓
제작자가 요청 수락
↓
assignee 배정
↓
상태: IN_PROGRESS
↓
에셋 제작 후 업로드
↓
완성 게시글과 요청글 연결
↓
상태: DONE
↓
완료 요청은 아카이브 처리

7.5 팀 프로젝트 전환 후 플로우

관리자가 사용자에게 teamId 배정
↓
사용자가 게시글 작성
↓
게시글에 userId와 teamId 함께 저장
↓
팀 전용 게시글은 같은 teamId 사용자만 조회 가능
↓
팀별 포트폴리오 또는 팀별 에셋 목록 구성 가능

8. 다음에 해야 할 것

각 도메인별 역할이 어느 정도 나왔으니, 일단 각 파트별 업무 내용을 대략적으로 Notion에 올려두면 좋을 것 같다. 그 다음 각자 하고 싶은 파트를 적어놓고, 업무량이 너무 몰리지 않게 조정하면 될 것 같다.

합의가 안 되면 가위바위보를 하든 랜덤으로 정해도 될 것 같다. 다만 프론트, 인프라, 파일, 검색 쪽은 업무량이 많을 수 있어서 혼자 맡기기보다는 보조 인원을 두는 게 좋을 것 같다.

9. 정리

- MVP는 작게 가되, 확장 가능성은 고려하자.
- 개인 단위에서 팀 단위로 바뀔 가능성을 미리 생각하자.
- 카테고리 분류 문제는 생각보다 어렵다.

- 태그 검색과 AI 추천이 현실적인 해결책이 될 수 있다.
- 요청 게시판은 GitHub Issue 처럼 상태 관리와 assignee 개념이 필요하다.
- 도메인별 구조로 나누되, 공통 규칙은 꼭 맞추자.
- 프론트와 인프라는 업무량이 클 수 있으니 역할 조정이 필요하다.

그래서 다음 전체 회의에서는 이 내용을 기준으로 어디까지 MVP로 가져갈지, 그리고 각 도메인을 누가 맡을지 정하면 될 것 같다.